



UNIDADE III

Desenvolvimento Web com .NET

Prof. Dr. Tarcísio Peres

Integração front-end e API

- React constrói a interface e responde às ações do usuário no navegador.
- ASP.NET Core expõe endpoints, aplica regras e acessa dados persistidos.
- Node sustenta as ferramentas usadas para desenvolver e empacotar o front-end.
- O navegador chama a API por HTTP e recebe dados, normalmente, em JSON.
- Contratos estáveis permitem que interface e serviço evoluam com autonomia.
- Regras críticas devem ser confirmadas pela API, não apenas pela interface.

React

- React organiza a interface em componentes pequenos e reutilizáveis.
- Um componente pode representar filtro, card, paginação ou página inteira.
- Propriedades levam dados e ações dos componentes pais para os filhos.
- useState mantém a memória local, como filtros, página e resultados.
- Quando o estado muda, React atualiza a parte necessária da interface.
- TypeScript explicita o formato esperado para dados como PetResponse.

Node, npm e Vite

- Node executa JavaScript fora do navegador e sustenta ferramentas do projeto.
- npm instala dependências e organiza scripts declarados em package.json.
- npm run dev inicia o servidor de desenvolvimento configurado pelo Vite.
- Vite entrega a aplicação em http://localhost:5173 e atualiza a tela rapidamente.
- Vite processa módulos e TypeScript e prepara os arquivos para publicação.
- Node não substitui ASP.NET Core; seu papel aqui é apoiar o front-end.

Back-end x front-end

- PetMatch.Web concentra a API, regras, SQLite, MVC e documentação OpenAPI.
- O back-end valida requisições e mantém a consistência dos dados persistidos.
- PetMatch.Frontend concentra React, TypeScript, componentes e estilos.
- O front-end transforma dados da API em filtros, cards e páginas interativas.
- Os dois projetos usam ferramentas próprias, mas pertencem à mesma solução.
- O contrato HTTP conecta as camadas sem misturar suas responsabilidades.

Interatividade

Qual é o papel principal do Vite no cenário apresentado?

- a) Substituir o ASP.NET Core no back-end.
- b) Criar migrations do banco SQLite.
- c) Servir e empacotar a aplicação React durante o desenvolvimento.
- d) Gerar controllers MVC automaticamente.
- e) Executar validações finais da API.

Resposta

Qual é o papel principal do Vite no cenário apresentado?

- a) Substituir o ASP.NET Core no back-end.
- b) Criar migrations do banco SQLite.
- c) Servir e empacotar a aplicação React durante o desenvolvimento.
- d) Gerar controllers MVC automaticamente.
- e) Executar validações finais da API.

CORS

- Uma origem é definida pela combinação de protocolo, domínio e porta.
- localhost:5173 e localhost:5213 são origens diferentes para o navegador.
- CORS informa quais origens podem chamar a API a partir de uma página web.
- A política do back-end pode liberar a origem, os métodos e os cabeçalhos necessários.
- Sem permissão, o navegador pode bloquear a resposta mesmo com a API ativa.
- CORS não substitui autenticação, autorização nem validação no servidor.

Proxy de desenvolvimento

- O proxy do Vite recebe caminhos do front-end e os encaminha ao back-end.
- Uma chamada a `/api/pets` pode seguir para `http://localhost:5213/api/pets`.
- O código React usa caminhos relativos em vez de repetir URLs e portas.
- O proxy também pode encaminhar `/Pets` para páginas MVC já existentes.
- Os caminhos `/css` e `/js` preservam os recursos usados por essas páginas.
- O proxy simplifica o desenvolvimento, mas não altera o contrato da API.

Cliente HTTP no front-end

- Um módulo como `petmatchClient.ts` centraliza as chamadas feitas com `fetch`.
- Funções como `listarPets` e `obterPet` escondem a montagem dos endpoints.
- `PetResponse` descreve um pet recebido da API com tipos explícitos.
- `PagedResponse<T>` descreve itens, página, tamanho e totais da resposta.
- O cliente verifica `response.ok` e transforma falhas em erros compreensíveis.
- Os componentes consomem funções prontas, sem espalhar URLs pela interface.

Estado, efeito e renderização

- useState guarda pets, filtros, página, carregamento e mensagem de erro.
- useEffect inicia a busca quando página ou filtros usados na consulta mudam.
- Antes da chamada, a tela ativa loading e limpa o erro anterior.
- No sucesso, a resposta atualiza os itens e a quantidade total de páginas.
- Ao finalizar, loading volta a false, com sucesso ou falha na requisição.
- A renderização distingue carregamento, erro, lista vazia e dados disponíveis.

Roteamento React

- BrowserRouter fornece o contexto de navegação à aplicação React.
- Routes e Route associam cada caminho a um componente de página.
- A rota / apresenta a lista de pets com filtros e paginação.
- A rota /pets/:id apresenta o detalhe do pet identificado na URL.
- useParams lê o id, e o cliente HTTP consulta /api/pets/{id}.
- Link navega entre páginas React sem confundir rotas de tela com rotas da API.

Lista/detalhe

- A lista favorece descoberta com filtros, cards, comparação e paginação.
- Cada card resume o pet e oferece a ação Ver detalhes.
- A página de detalhe busca um único item a partir do identificador da rota.
- O detalhe oferece mais espaço para descrição, características e ações.
- As duas páginas tratam carregamento, erro e ausência de dados com clareza.
- Estados visuais consistentes orientam a navegação e reforçam a confiança.

Interatividade

Por que o front-end React usa um cliente HTTP separado, como `petmatchClient.ts`?

- a) Para impedir o uso de componentes.
- b) Para centralizar chamadas à API e evitar repetição de URLs.
- c) Para substituir o roteamento React.
- d) Para remover a necessidade de CORS.
- e) Para salvar dados diretamente no SQLite.

Resposta

Por que o front-end React usa um cliente HTTP separado, como `petmatchClient.ts`?

- a) Para impedir o uso de componentes.
- b) Para centralizar chamadas à API e evitar repetição de URLs.**
- c) Para substituir o roteamento React.
- d) Para remover a necessidade de CORS.
- e) Para salvar dados diretamente no SQLite.

Autenticação, autorização e validação

- Autenticação confirma quem enviou a requisição e estabelece uma identidade confiável.
- Autorização decide quais recursos e ações essa identidade pode acessar.
- Validação verifica se formato, conteúdo e regras dos dados são aceitáveis.
- Controles complementares: um usuário autorizado ainda pode enviar dados inválidos.
- Sequência mental: identificar → verificar permissão → validar → processar.

ASP.NET Identity integra identidade e persistência

- Infraestrutura: usuários, hash de senha, roles, claims, logins, tokens internos e cookies.
- UserManager cria e consulta usuários; SignInManager executa entrada e saída.
- ApplicationUser pode herdar de IdentityUser e incluir campos do domínio, como nome e cidade.
- EF Core: AddEntityFrameworkStores conecta os serviços ao contexto da aplicação.
- Banco de dados: IdentityDbContext cria tabelas próprias para usuários, roles, claims, logins e vínculos.

Cookies e JWT atendem a contextos diferentes

- Cookie: mantém sessão no navegador e é enviado automaticamente ao mesmo site.
- Uso em navegação: páginas MVC, formulários e navegação web no mesmo back-end.
- JWT: token textual enviado como Authorization: Bearer em clientes independentes.
- Uso em APIs: aplicativos móveis, integrações e serviços distribuídos.
- Regra comum: o servidor deve validar cookie ou JWT em cada requisição protegida.
- Decisão: escolha pelo contexto do cliente, não por uma suposta superioridade universal.

Roles expressam papéis amplos de acesso

- Role: agrupa usuários que exercem uma função semelhante no sistema.
- Exemplos: Administrador e Voluntario.
- Atribuição: roles podem ser criadas no seed e associadas ao usuário no cadastro.
- Proteção declarativa: [Authorize(Roles = "Voluntario,Administrador")].
- Bom uso: regras descritas como pertencimento a um grupo de acesso.
- Limite: uma role não prova, sozinha, que o usuário é responsável por um recurso específico.

Claims permitem decisões mais granulares

- Claim: declaração do tipo chave-valor associada à identidade autenticada.
- Exemplos: cidade, nome, identificador, permissão específica ou vínculo institucional.
- Granularidade: expressa características e permissões mais específicas que roles.
- Políticas: podem exigir claims determinadas para liberar uma operação.
- Perfil: as mesmas declarações podem alimentar informações exibidas pela aplicação.
- Recurso: regras de propriedade ainda devem conferir o vínculo entre usuário e dado acessado.

Interatividade

Qual alternativa diferencia corretamente autenticação e autorização?

- a) Autenticação valida dados; autorização cria o banco.
- b) Autenticação identifica o usuário; autorização decide o que ele pode acessar.
- c) Autenticação só existe em React; autorização só existe em MVC.
- d) Autorização substitui login e senha.
- e) Autenticação e autorização são sempre a mesma coisa.

Resposta

Qual alternativa diferencia corretamente autenticação e autorização?

- a) Autenticação valida dados; autorização cria o banco.
- b) Autenticação identifica o usuário; autorização decide o que ele pode acessar.
- c) Autenticação só existe em React; autorização só existe em MVC.
- d) Autorização substitui login e senha.
- e) Autenticação e autorização são sempre a mesma coisa.

O pipeline autentica antes de autorizar

- UseAuthentication() lê a credencial e preenche HttpContext.User.
- UseAuthorization() aplica atributos e políticas à identidade já reconhecida.
- Ordem obrigatória: autenticar primeiro; autorizar depois.
- [Authorize] depende desse contexto para proteger controllers, actions e endpoints.
- 401 Unauthorized: a requisição não apresenta autenticação válida.
- 403 Forbidden: o usuário está autenticado, mas não possui a permissão exigida.

A proteção real permanece no servidor

- Acesso público: listagem e detalhes podem ser abertos a visitantes.
- Acesso protegido: cadastro e edição exigem login e role adequada.
- Interface: ocultar botões e redirecionar melhoram a experiência do usuário.
- Servidor: controllers e APIs devem repetir a verificação antes de alterar dados.
- Motivo: endpoints também podem ser chamados por scripts, testes ou outros clientes.
- Resultado: nenhuma ação sensível depende apenas do que o React mostra na tela.

A SPA trata autenticação como estado global

- Inicialização: a SPA consulta `/api/auth/me` para descobrir se existe sessão válida.
- AuthContext: concentra usuário, carregamento, `isAuthenticated`, `login`, `logout` e atualização.
- Fonte única: navegação, perfil e rotas protegidas consultam o mesmo estado.
- Login: atualiza a sessão no servidor e a interface para o estado autenticado.
- Logout: encerra a sessão e devolve a aplicação ao estado anônimo.
- Estados previstos: carregando, autenticado, anônimo, sessão expirada e falha de rede.

Cookie seguro reduz a exposição na SPA

- credentials: “include” envia e recebe o cookie nas chamadas fetch previstas.
- HttpOnly impede que o JavaScript leia o cookie de autenticação.
- Secure restringe o transporte do cookie a conexões HTTPS.
- SameSite ajuda a controlar o envio do cookie em contextos entre sites.
- Evite guardar tokens de autenticação em localStorage ou sessionStorage.
- APIs: retorne 401 ou 403 à SPA, em vez de redirecionar a chamada para uma página HTML.

Rotas React organizam a experiência de acesso

- Carregando: mostrar uma espera enquanto a sessão é verificada.
- Anônimo: redirecionar para /entrar e preservar o destino solicitado.
- Autenticado: renderizar a página ou o Outlet protegido.
- Experiência: ProtectedRoute evita telas incoerentes durante a navegação.
- Segurança: a rota React não substitui [Authorize] nem as políticas do servidor.
- Teste final: verificar acesso anônimo, retorno após login, logout e sessão expirada.

Interatividade

Em uma aplicação React integrada a ASP.NET Identity por cookie, por que usar credentials: “include” nas chamadas fetch?

- a) Para salvar o cookie no localStorage.
- b) Para enviar e receber cookies de autenticação nas requisições.
- c) Para desativar autorização no servidor.
- d) Para converter cookies em JWT.
- e) Para impedir chamadas à API.

Resposta

Em uma aplicação React integrada a ASP.NET Identity por cookie, por que usar credentials: “include” nas chamadas fetch?

- a) Para salvar o cookie no localStorage.
- b) Para enviar e receber cookies de autenticação nas requisições.**
- c) Para desativar autorização no servidor.
- d) Para converter cookies em JWT.
- e) Para impedir chamadas à API.

ATÉ A PRÓXIMA!